

From C++ to Objective-C

version 1.11-en

Pierre CHATELIER
e-mail: pierre.chatelier@club-internet.fr

Copyright © 2005, 2006, 2007 Pierre CHATELIER

English adaptation : Aaron VEGH

Document revisions available at :
<http://ktd.club.fr/programmation/objective-c.php>

This document is also available in french
Ce document est aussi disponible en français

With special thanks to: For their attentive reading and many helpful comments, I would like to thank Pascal BLEUYARD, Jérôme CORNET, François DELOBEL and Jean-Daniel DUPAS, whose help was important in making this work the best possible. Jack NUTTING, Ben RIMMINGTON and Mattias ARRELID have also provided many feedback. Jonathon MAH has been particularly implied in bringing a lot of very judicious corrections.

Contents

Table of contents	2
Introduction	5
1 Objective-C and Cocoa	6
2 A short history of Objective-C	6
3 Syntax overview	6
3.1 Keywords	6
3.2 Comments	7
3.3 Mixing up code and declarations	7
3.4 New types and values	7
3.4.1 BOOL, YES, NO	7
3.4.2 nil, Nil and id	7
3.4.3 SEL	7
3.5 Class names: why NS?	7
3.6 Differencing functions and methods	8
4 Organization of source code: .h and .m files, inclusion	8
5 Classes and objects	9
5.1 Root class, type id, nil and Nil values	9
5.2 Class declaration	9
5.2.1 Attributes and methods	9
5.2.2 Forward declarations: @class, @protocol	10
5.2.3 public, private, protected	11
5.2.4 static attributes	11
5.3 Methods	11
5.3.1 Prototype and call, instance methods, class methods	11
5.3.2 this, self and super	12
5.3.3 Accessing instance variables inside a method	13
5.3.4 Prototype id and signature, overloading	13
5.3.5 Pointer to member function: Selector	14
5.3.6 Default values of parameters	16
5.3.7 Variable number of arguments	16
5.3.8 Anonymous arguments	16
5.3.9 Prototype modifiers (const, static, virtual, ‘= 0’, friend, throw)	17
5.4 Messages and transmission	17
5.4.1 Sending a message to nil	17
5.4.2 Delegating a message to an unknown object	17
5.4.3 Forwarding: handling an unknown message	17
5.4.4 Downcasting	18
6 Inheritance	19
6.1 Simple inheritance	19
6.2 Multiple inheritance	19
6.3 Virtuality	19
6.3.1 Virtual methods	19
6.3.2 Silent redefinition of virtual methods	19
6.3.3 Virtual inheritance	19
6.4 Protocols	20
6.4.1 Formal protocol	20
6.4.2 Informal protocol	21
6.4.3 Message qualifiers for distant objects	22

6.5	Class categories	22
6.6	Joint use of protocols, categories, subclassing:	23
7	Instantiation	24
7.1	Constructors, initializers	24
7.1.1	Distinction between <i>allocation</i> and <i>initialization</i>	24
7.1.2	Using <code>alloc</code> and <code>init</code>	24
7.1.3	Example of a correct initializer	25
7.1.4	Initialization failure	26
7.1.5	Default constructor : designated initializer	26
7.1.6	List of initialization and default value of instance data	28
7.1.7	Virtual constructor	28
7.1.8	Class constructors	28
7.2	Destructors	28
7.3	Copy operators	29
7.3.1	Classical cloning, <code>copy</code> and <code>copyWithZone:</code>	29
7.3.2	<code>NSCopyObject()</code>	30
7.3.3	Dummy-cloning, mutability, <code>mutableCopy</code> and <code>mutableCopyWithZone:</code>	31
8	Memory management	33
8.1	<code>new</code> and <code>delete</code>	33
8.2	Reference counting	33
8.3	<code>alloc</code> , <code>copy</code> , <code>mutableCopy</code> , <code>retain</code> , <code>release</code>	33
8.4	<code>autorelease</code>	33
8.4.1	<code>autorelease</code>	33
8.4.2	The autorelease pool	35
8.4.3	Using several autorelease pools	35
8.4.4	Caution with <code>autorelease</code>	35
8.4.5	<code>autorelease</code> and <code>retain</code>	35
8.4.6	Convenience constructor, virtual constructor	36
8.4.7	Setter	37
8.4.8	Getter	39
8.5	Retain cycles	40
8.6	Garbage collector	41
9	Exceptions	42
10	Strings in Objective-C	44
10.1	The only static objects in Objective-C	44
10.2	<code>NSString</code> and encodings	44
10.3	Description of an object, <code>%@</code> format extension, <code>NSString</code> to C string	44
11	STL and Cocoa	45
11.1	Containers	45
11.2	Iterators	45
11.2.1	Classical enumeration	45
11.2.2	Fast enumeration	46
11.3	Functors (function objects)	46
11.3.1	Using selectors	46
11.3.2	IMP caching	46
11.4	Algorithms	46

12 C++ specific features	47
12.1 References	47
12.2 Inlining	47
12.3 Templates	47
12.4 Operators overloading	47
12.5 Friends	47
12.6 const methods	47
12.7 List of initialization in the constructor	47
13 RTTI (Run-Time Type Information)	48
13.1 class, superclass, isMemberOfClass, isKindOfClass	48
13.2 conformsToProtocol	48
13.3 respondsToSelector, instancesRespondToSelector	48
13.4 Strong typing or weak typing with id	49
14 Objective-C++	49
15 Objective-C 2.0	50
Conclusion	51
References	51
Document revisions	52
Index	53

Introduction

This document is designed to act as a bridge between C++ and Objective-C. Many texts exist to teach the object model through Objective-C, but to my knowledge, none target advanced C++ developers wanting to compare these concepts with their own knowledge. At first, the Objective-C language seems to be an obstacle rather than a boost for **Cocoa** programming (cf. section 1 on the following page): it was so different that I couldn't get into it. It took me some time to appreciate its challenges, and understand the many helpful concepts it supplies. This document is not a tutorial, but a quick reference, to these concepts. I hope that it will be useful in preventing a developer from either abandoning Objective-C or misusing its features, because of a misunderstanding of the language. This document does not claim to be a *full* reference, but a *quick* one. For detailed explanations of a concept, be sure to read a specialized Objective-C manual [1].

1 Objective-C and Cocoa

An initial distinction should be made : **Objective-C** is a language, while **Cocoa** is a set of classes that contribute to native MacOS X programming. Theoretically, it is possible to use Objective-C without Cocoa: there is a gcc front-end. But under MacOS X, both are almost inseparable, as most of the classes supplied by the language are part of Cocoa.

More precisely, Cocoa is the implementation by Apple, for MacOS X, of the OpenStep standard, originally published in 1994. It consists of a developer framework based upon Objective-C. The GNUstep project [3] is another implementation, which is free. Its goal is to be as portable as possible on most Unix systems, and is still under development.

2 A short history of Objective-C

It is hard to give a precise date of birth for a language, owing to the fact that there is some time between first steps, improvements, standardisation and official announcement. However, a rough history is given in Figure 1 to get a quick look at Objective-C amongst its ancestors and “challengers”.

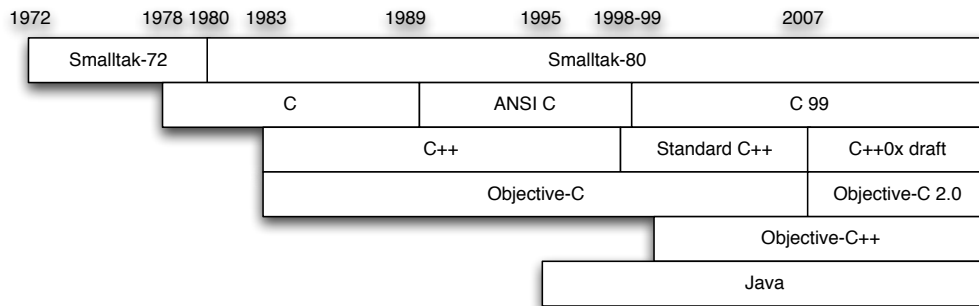


Figure 1: Timeline of Java, C, C++ and Objective-C

Smalltalk-80 is one the first “real” object languages. C++ and Objective-C are two different branches that build a superset of the C language. Objective-C is very close to Smalltalk in terms of syntax and dynamism, while C++ is much more static, with the goal of having better run-time performance. Java targets a C++ audience, but is also very inspired by Smalltalk for its object model. That’s why, despite this document’s title, many references are made to Java.

Objective-C++ is a kind of merge between Objective-C and C++. It is already usable, but some behaviours are still not perfect. The goal of Objective-C++ is to mix up the syntaxes of Objective-C and C++ to benefit from the best features of both worlds (cf. section 14 on page 49).

3 Syntax overview

3.1 Keywords

Objective-C is a superset of the C language. Like with C++, a well-written C program should be compile-able as Objective-C, as long as it is not using some of the bad practices allowed by C. Objective-C has only added some concepts and their associated keywords. To avoid conflicts, these keywords begin with the @ (at) character. Here is the (short) exhaustive list: @class, @interface, @implementation, @public, @private, @protected, @try, @catch, @throw, @finally, @end, @protocol, @selector, @synchronized, @defs, @encode. Let us also quote the values nil and Nil, the types id, SEL and BOOL, the boolean values being YES and NO. At last, six more keywords are available in a particular context, and are not reserved outside: in, out, inout, bycopy, byref, oneway. They can be met when defining protocols (cf. section 6.4.3 on page 22).

There is an easy confusion between the language keywords and some methods inherited from the root class NSObject (the mother of every class, cf. section 5.1 on page 9). For instance, the similar-

looking “keywords” for memory management, named `alloc`, `retain`, `release` and `autorelease`, are in fact methods of `NSObject`. The words `super` and `self` (cf. section 5.3.1 on page 11), could also be understood as keywords, but `self` is in fact a hidden parameter to each method, and `super` an instruction asking the compiler to use `self` differently. However, the confusion between these false keywords and the true ones will not likely prove problematic in normal use.

Not all the keywords of Objective-C are presented in this document. `@synchronized`, `@defs` and `@encode` have been put aside because they are not relevant for comparing with the C++ language: they have no equivalent there. They are not required to translate a program from standard C++ to Objective-C.

3.2 Comments

The comments `/* ... */` and `//` are allowed.

3.3 Mixing up code and declarations

Like in C++, it is possible to insert the declaration of a variable in the middle of a block of instructions.

3.4 New types and values

3.4.1 BOOL, YES, NO

In C++, the boolean type is `bool`. In Objective-C, it is `BOOL`, which can be set to `YES` or `NO`.

3.4.2 nil, Nil and id

These three keywords are explained later in the document, but briefly:

- Every object is of type `id`. This is a tool for weak-typing;
- `nil` is the equivalent of `NULL` for a pointer to an object. `nil` and `NULL` should not be interchangeable.
- `Nil` is the equivalent of `nil` for a class pointer. In Objective-C, a class is an object (it is the instance of a meta-class).

3.4.3 SEL

The `SEL` type can store selectors values (method identifiers). These values can be computed by a call to `@selector`. It is the Objective-C implementation for pointers on member functions. See section 5.3.4 on page 13 for more details.

3.5 Class names: why NS?

In this document, almost all class names begin with *NS*, like *NSObject* or *NSString*. The reason is simple: they are Cocoa classes, and most Cocoa classes begin with *NS* since they were initiated under NeXTStep.

It is a common practice to use a prefix to identify the origin of a class.

3.6 Differencing functions and methods

Objective-C is not a language with “function calls using square brackets”. This would be a legitimate thought when observing code like that :

```
[object doSomething];
```

instead of

```
object.doSomething();
```

But in fact, Objective-C is a superset of C, so that *functions* match the same syntax and semantics as C for declaration, implementation and call. On the contrary, *methods*, which do not exist in C, have a special syntax, which includes square brackets. Moreover, the difference is not only in the syntax, but also the meaning. This is detailed further in Section 5.2 on the next page: this is not a method call, this is sending a message. This is not just a simple academic distinction; it has implications on the mechanism of Objective-C. The syntax is also more readable, especially with nested calls (cf. section 5.3.1 on page 11).

4 Organization of source code: *.h* and *.m* files, inclusion

Like in C++, it is useful to split the code between interface and implementation for each class. Objective-C uses *.h* files for headers, and *.m* files for the code; *.mm* files are used for Objective-C++ (see Section 14 on page 49). Objective-C introduces the `#import` directive to replace `#include`. Indeed, every C header should use compilation guards to prevent multiple inclusions. This is automatic when using `#import`. Below is a typical interface/implementation example. The Objective-C syntax is explained later.

C++	
<pre>//In file Foo.h #ifndef __FOO_H__ //compilation guard #define __FOO_H__ // class Foo { ... }; #endif</pre>	<pre>//In file Foo.cpp #include "Foo.h" ...</pre>
Objective-C	
<pre>//In file Foo.h //class declaration, different from //the "interface" Java keyword @interface Foo : NSObject { ... } @end</pre>	<pre>//In file Foo.m #import "Foo.h" @implementation Foo ... @end</pre>

5 Classes and objects

Objective-C is an object-oriented language: it manages classes and objects. Objective-C uses a strict object model, unlike C++ which has many discrepancies against the ideal object model. For instance, in Objective-C, classes are objects and can be dynamically managed: it is possible to add classes at run-time, create instances based on the name of the class, ask a class for its methods, and so on. This is much more powerful than C++ RTTI (cf. section 13 on page 48), which have been added to a very “static” language. Discouraging RTTI use is common since the results may depend on the compiler itself and lack portability.

5.1 Root class, type `id`, `nil` and `Nil` values

In an object-oriented language, each program makes use of a set of classes. Unlike C++, Objective-C defines a root class. Every new class should be a descendant of the root class. In Cocoa, that class is `NSObject`, and it provides a huge number of facilities for the run-time system. The root class notion is not specific to Objective-C; it’s related to the object model. Smalltalk and Java make use of a root class, while C++ does not.

Strictly speaking, every object should be of type `NSObject`, and every pointer to an object could be declared as `NSObject*`. In fact, one can use the type `id` instead. This is a short and handy way to declare a pointer to any object, and provides dynamic type-checking instead of static type-checking. It is very useful for some weak typing on generic methods. Please note that a null pointer to an object should be set to `nil`, not `NULL`. These values are not interchangeable. A normal C pointer can be set to `NULL`, but `nil` was introduced in Objective-C for pointers to objects. In Objective-C, classes are also objects (meta-class instances), and it is possible to declare a pointer to a class. Their null value is `Nil`.

5.2 Class declaration

It is hard to show with a single example all the differences between Objective-C and C++ for class declaration and implementation. Syntax and concepts are interleaved and require explanation. In the following, the differences are exposed sequentially and specifically.

5.2.1 Attributes and methods

In Objective-C, attributes are called *instance data*, and member functions are called *methods*.

C++	Objective-C
<pre>class Foo { double x; public: int f(int x); float g(int x, int y); }; int Foo::f(int x) {...} float Foo::g(int x, int y) {...}</pre>	<pre>@interface Foo : NSObject { double x; } -(int) f:(int)x; -(float) g:(int)x :(int)y; @end @implementation Foo -(int) f:(int)x {...} -(float) g:(int)x :(int)y {...} @end</pre>

In C++, attributes and methods are declared together inside the braces of the class. Method implementation syntax is similar to C, with the addition of the scope resolution operator (`Foo::`).

In Objective-C, attributes and methods cannot be mixed. The attributes are declared in braces, the methods follow. Their implementation lies in an `@implementation` block.

This is a major difference with C++, since some methods can be implemented without being exposed in the interface. This is detailed later. Briefly, this is a way to clean up header files by removing unnecessary declarations (“private” methods, and silently redefined virtual methods like destructors). Please see Section 6.3.2 on page 19 for further explanations.

Instance methods are prefixed by the minus symbol “-”, and class methods by the plus symbol “+”(cf. section 5.3.9 on page 17); this symbol has nothing to do with the UML notation and the meaning *public* or *private*. The type of the parameters are enclosed in parenthesis, and the parameters are separated by the symbol “:”. Please see Section 5.3.1 on the next page for further explanations on the syntax of prototypes.

In Objective-C, there is no need for a semi-colon at the end of a class declaration. Also note that the keyword to declare a class is `@interface` and not `@class`. The keyword `@class` is only used in forward declarations (cf. section 5.2.2 on the current page). Finally, if there is no instance data in a class, the braces, which would enclose nothing, can be omitted.

5.2.2 Forward declarations: `@class`, `@protocol`

To avoid cyclic dependencies in header files, the C language supports the *forward declaration*, that allows the coder to declare a class when the only required knowledge is its existence and not its structure. In C++, the keyword `class` is used; in Objective-C, it is `@class`. The keyword `@protocol` can also be used to anticipate the declaration of a protocol (cf. section 6.4 on page 20).

C++	
<pre>//In file Foo.h #ifndef __FOO_H__ #define __FOO_H__ class Bar; //forward declaration class Foo { Bar* bar; public: void useBar(void); }; #endif</pre>	<pre>//In file Foo.cpp #include "Foo.h" #include "Bar.h" void Foo::useBar(void) { ... }</pre>
Objective-C	
<pre>//In file Foo.h @class Bar; //forward declaration @interface Foo : NSObject { Bar* bar; } -(void) useBar; @end</pre>	<pre>//In file Foo.m #import "Foo.h" #import "Bar.h" @implementation Foo -(void) useBar { ... } @end</pre>

5.2.3 public, private, protected

One major feature of the object model is data encapsulation, which limits the visibility of data to some parts of the code, in order to ensure its integrity.

C++	Objective-C
<pre>class Foo { public: int x; int apple(); protected: int y; int pear(); private: int z; int banana(); };</pre>	<pre>@interface Foo : NSObject { @public int x; @protected: int y; @private: int z; } -(int) apple; -(int) pear; -(int) banana; @end</pre>

In **C++**, attributes and methods can belong to a **public**, **protected** or **private** scope. The default mode is **private**.

In **Objective-C**, only the instance data can be **public**, **protected** or **private**, and the default visibility is **protected**. Methods can only be **public**. However, it is possible to mimic the private mode, by implementing some methods in the **@implementation**, without declaring them in the **@interface**, or using the notion of class category (cf. section 6.5 on page 22). Implementing a method without a previous declaration is a special property of Objective-C and has a specific purpose, as explained in Section 6.3.2 on page 19.

Inheritance cannot be tagged **public**, **protected** or **private**. The only way is **public**. Inheritance in Objective-C looks more like Java than C++ (section 6 on page 19).

5.2.4 static attributes

It's not possible in Objective-C to declare a class data attribute (**static** as in C++). However, it is possible to do the same thing in a different way: use a global variable in the implementation file (optionally with the **static C** keyword to limit the scope). The class can then use accessors on it (with class methods or normal methods), and its initialization can be done in the **initialize** method of the class (cf. section 7.1.8 on page 28).

5.3 Methods

The syntax for a method in Objective-C is different from the syntax for common C functions. This section aims to describe this syntax and adds some information on the underlying message sending principle.

5.3.1 Prototype and call, instance methods, class methods

- A method is prefixed by “-” if it is an instance method (common case), or by “+” if it is a class method (**static** in C++). This symbol is in no way linked to the UML notation for *public* or *private*. Methods are always **public** in Objective-C;
- the types for the return value, or for the parameters, are enclosed by parentheses;

- the parameters are separated by a colon “:”;
- the parameters can be associated with a **label**, a name specified just before “:”; **the label is then part of the name of the method and modifies it**. That makes calls to this function particularly readable. In fact, the label usage should be systematic. Note that the first parameter cannot have a label – indeed, its label is already the name of the method;
- a method name can be the same as an attribute name, without conflict. This is very useful for getters (cf. section 8.4.8 on page 39).

C++
<pre>//prototype void Array::insertObject(void *anObject, unsigned int atIndex) //use with a "shelf" instance of the Array class and a "book" object shelf.insertObject(book, 2);</pre>

Objective-C
Without label (direct translation from C++)
<pre>//prototype //The method is named "insertObject::", the colon being used to separate //the parameters (it is not a scope resolution operator like in C++) -(void) insertObject:(id)anObject:(unsigned int)index //use with a "shelf" instance of the Array class and a "book" object [shelf insertObject:book:2];</pre>
With label
<pre>//prototype. The "index" parameter is now labeled "atIndex" //The method is now named "insertObject:atIndex:" //The call can easily be read like a sentence -(void) insertObject:(id)anObject atIndex:(unsigned int)index //use with a "shelf" instance of the Array class and a "book" object [shelf insertObject:book:2]; //Error ! [shelf insertObject:book atIndex:2]; //OK</pre>

Please note that the syntax with square brackets should not be read as *calling the method* `insertObject` of object “shelf” but rather *sending the message* `insertObject` to object “shelf”. This is what makes Objective-C. One can send any message to any target. If it is not able to handle the message, it can ignore it (an exception is raised but the program does not abort). If, at the time it receives the message, a target is able to handle it, then the matching method will be called. The compiler is able to raise a warning if a message is sent to a class for which it is known that no matching method exists. This is not considered as an error thanks to the *forwarding* facility (cf. section 5.4.3 on page 17). If a target is only known under the type `id`, no warning will be made at compile time; a potential error would be raised at run-time.

5.3.2 `this`, `self` and `super`

There are two particular targets for a message: `self` and `super`. `self` is the current object (like `this` in C++), `super` is the parent class. The keyword `this` does not exist in Objective-C. It is replaced by `self`.

NB: `self` is not a real keyword, it is a hidden parameter that each method receives, the value of which is the current object. Its value can be changed, unlike C++’s `this` keyword. However, this is only useful in constructors (cf. section 7.1 on page 24).

5.3.3 Accessing instance variables inside a method

Like in C++, an Objective-C method can access the instance variables of its current object. The optional `this->` in C++ can be written `self->`.

C++	Objective-C
<pre>class Foo { int x; int y; void f(void); }; void Foo::f(void) { x = 1; int y; //making ambiguity with this->y y = 2; //using local y this->y = 3; //disambiguating }</pre>	<pre>@interface Foo : NSObject { int x; int y; } -(void) f; @end @implementation Foo -(void) f { x = 1; int y; //making ambiguity with this->y y = 2; //using local y self->y = 3; //disambiguating } @end</pre>

5.3.4 Prototype id and signature, overloading

A function is a part of code that can be referenced, for instance to use as function pointers, or functors. Moreover, even if the name of the function is a good candidate for a unique id, one must take care when using overloading. C++ and Objective-C use opposite ways to differentiate prototypes. The first is based on parameters types, while the second is based on parameter labels.

In C++, two functions can have the same name as long as their parameters have different types. When using methods, the `const` option is also discriminative.

C++
<pre>int f(int); int f(float); //OK, float is different from int class Foo { public: int g(int); int g(float); //OK, float is different from int int g(float) const; //OK, const is discriminative }; class Bar { public: int g(int); //OK, we are in Bar::, different from Foo:: }</pre>

In Objective-C, all functions are C functions: they cannot be overloaded. However, methods use a different syntax, and can be differentiated thanks to parameter labels.

Objective-C
<pre> int f(int); int f(float); //Error : C functions cannot be overloaded @interface Foo : NSObject { } -(int) g:(int) x; -(int) g:(float) x; //Error : this method is not different // from the previous one (no label) -(int) g:(int) x :(int) y; //OK : two anonymous labels -(int) g:(int) x :(float) y; //Error : not different from the //previous one -(int) g:(int) x andY:(int) y; //OK : second label is "andY" -(int) g:(int) x andY:(float) y; //Error : not different from the //previous one -(int) g:(int) x andAlsoY:(int) y; //OK : second label is //"andAlsoY", different from "andY" @end </pre>

The label-based identifying method is useful for expressing the exact “name” of the function, as stated below.

<pre> @interface Foo : NSObject {} //The method name is "g" -(int) g; //The method name is "g:" -(int) g:(float) x; //The method name is "g::" -(int) g:(float) x :(float) y; //The method name is "g:andY:" -(int) g:(float) x andY:(float) y; //The method name is "g:andZ:" -(int) g:(float) x andZ:(float) z @end </pre>
--

Clearly, two Objective-C methods are differentiated by *labels* instead of *types*. This is a way to replace “pointers on member functions”, expressed by *selectors*, as explained in section 5.3.5 on the current page.

5.3.5 Pointer to member function: Selector

In Objective-C, the *methods* have a particular syntax using parentheses and labels. It is not possible to declare *functions* with this syntax. The notion of pointer to *functions* is the same in C and Objective-C. It is only for pointers on *methods* where a difference occurs.

In C++, the syntax is hard but consistent with the C language : the pointer focus is on types.

C++
<pre> class Foo { public: int f(float x) {...} }; Foo bar int (Foo::*p_f)(float) = &Foo::f; //Pointer to Foo::f (bar.*p_f)(1.2345); //calling bar.f(1.2345); </pre>

In Objective-C, a new type has been introduced. A pointer to a method is called a *selector*. Its type is `SEL` and its value is computed using `@selector` on the exact name of the method (with labels of parameters). Calling the method can be done using the class `NSInvocation`. Most of the time, utility methods of the `performSelector:` family (inherited from `NSObject`) are handier, but a little restrictive. The simplest three ones are:

```

-(id) performSelector:(SEL)aSelector;
-(id) performSelector:(SEL)aSelector withObject:(id)anObjectAsParameter;
-(id) performSelector:(SEL)aSelector withObject:(id)anObjectAsParameter
        withObject:(id)anotherObjectAsParameter;

```

The returned value is the same as that of the called method. For methods which have parameters that are not objects, one should generally use wrapper classes like `NSNumber`, that can provide `float`, `int`, and so on. The `NSInvocation` class, which is more general and powerful, can also be used (see the documentation).

According to what has been said, nothing prevents a method from being called on an object, even if the object does not implement it. In fact, the method is effectively triggered if the message is accepted. But an exception, that can be caught, is raised when the object does not know the method; the application is not interrupted. Moreover, it is possible to check that an object can trigger a method by calling `respondToSelector:`.

Finally, the value of `@selector()` is computed at compile-time, which does not slow down the code.

Objective-C

```
@interface Slave : NSObject {}
    -(void) readDocumentation:(Document*)document;
@end

//Let us suppose an array "array[]" of 10 slaves,
//and a document "document"

//Normal method call
for(i=0 ; i<10 ; ++i)
    [array[i] readDocumentation:document];

//Just for the example, try to use performSelector: instead
for(i=0 ; i<10 ; ++i)
    [array[i] performSelector:@selector(readDocumentation:)
                        withObject:document];

//The type of a selector is SEL
//The following version is not more efficient than the previous one,
//since @selector() is evaluated at compile-time
SEL methodSelector = @selector(readDocumentation:);
for(i=0 ; i<10 ; ++i)
    [slaves[i] performSelector:methodSelector withObject:document];

//for an object "foo" which type is not known (id)
//the test is not mandatory, but prevents an exception to be raised
//if the object has no readDocumentation: method.
if ([foo respondsToSelector:@selector(readDocumentation:)])
    [foo performSelector:@selector(readDocumentation:) withObject:document];
```

5.3.6 Default values of parameters

Objective-C does not allow a default value to be specified for the parameters of functions or methods. One should create as much functions as necessary when some parameters could be optional. In the case of constructors, one should use the notion of *designated initializer* (section 7.1.5 on page 26).

5.3.7 Variable number of arguments

Objective-C allows the use of methods with a variable number of arguments. Like in C, the syntax relies on using “...” as the last argument. This is rarely useful, even if many Cocoa methods are doing so. More details can be found in the Objective-C documentation.

5.3.8 Anonymous arguments

In C++, it is possible to give no name to a parameter in a prototype, since the type is sufficient to characterize the signature of a function. This is not possible in Objective-C.

5.3.9 Prototype modifiers (`const`, `static`, `virtual`, `'= 0'`, `friend`, `throw`)

In C++, a few modifiers can be added to the prototype of a function. None of them exist in Objective-C. Here is the list:

- `const`: a method cannot be set `const`. As a consequence, the `mutable` keyword cannot exist;
- `static`: making the difference between an instance method and a class method is done through the use of “-” or “+” in front of the prototype;
- `virtual`: Objective-C methods are virtual, so the keyword is useless. **Pure** virtual methods are implemented with a formal protocol (cf. section 6.4 on page 20);
- `friend`: there is no “friend” notion in Objective-C;
- `throw`: in C++, one can restrict a method to transmit some exceptions only. This is not possible in Objective-C.

5.4 Messages and transmission

5.4.1 Sending a message to `nil`

By default, it is legal to send a message (call a method) to `nil`. The message is just ignored. The code can be greatly simplified by reducing the number of tests usually made with the null pointer. GCC has an option to disable that handy behaviour, for extra optimizations.

5.4.2 Delegating a message to an unknown object

Delegation is common with user interface elements in Cocoa (tables, outlines...), taking advantage of the ability to send a message to an unknown object. An object can, for example, delegate some tasks to an assistant.

```
//This is a function to define an assistant
-(void) setAssistant:(id)slave
{
    [assistant autorelease]; //see the section about memory management
    assistant = [slave retain];
}
```

```
//the method performHardWork can use delegation
-(void) performHardWork:(id)task
{
    //the assistant is not known
    //we check that it can handle the message
    if ([assistant respondsToSelector:@selector(performHardWork:)])
        [assistant performHardWork:task];
    else
        [self findAnotherAssistant];
}
```

5.4.3 Forwarding: handling an unknown message

In C++, code cannot be compiled if a method is called on an object that does not implement it. In Objective-C, there's a difference: one can always send a message to an object. If it can't be handled at run-time, it will be ignored (and raise an exception); moreover, instead of ignoring it, it can forward the message to another object.

When the compiler is told the type of an object, it can detect if a message sending – method call – will fail, and raise a warning. However, this is not an error, since in this case, an alternative is

available. This second chance is represented by a call to the `forwardInvocation:` method, which can be redefined to redirect the message at the very last moment. This is obviously a method of `NSObject`, which does nothing by default. Here is another way to manage assistant objects.

```
-(void) forwardInvocation:(NSInvocation*)anInvocation
{
    //if we are here, that is because the object cannot handle
    //the message of the invocation
    //the bad selector can be obtained by sending "selector"
    //to the object "anInvocation"
    if ([anotherObject respondsToSelector:[anInvocation selector]])
        [anInvocation invokeWithTarget:anotherObject];
    else //do not forget to try with the superclass
        [super forwardInvocation:anInvocation];
}
```

Even if a message can be handled, at last, in a `forwardInvocation:`, and only there, a check based on `respondsToSelector:` will still return NO. Indeed, the `respondsToSelector:` mechanism is not designed to guess whether `forwardInvocation:` will work or not.

5.4.4 Downcasting

Downcasting is needed in C++ to call methods on a subclass, when only a parent class pointer is known, this is made possible using `dynamic_cast`. In Objective-C, this practice is not necessary, since a message can be sent to an object even if it seems that the object cannot handle it.

However, to avoid a compiler warning, one can simply *cast* the type of the object; there is no explicit *downcasting* operator in Objective-C, the traditional *cast* syntax of the C language can be used.

```
//NSMutableString is a subclass of NSString (string of characters)
//that allows mutating operations
NSMutableString* mutableString = ...initializing a mutable string...
NSString* string = mutableString; //storing in an NSString pointer
[string appendString:@"foo"]; //compiler warning
[(NSMutableString*)string appendString:@"foo"]; //no warning
[(id)string appendString:@"foo"]; //no warning
```

6 Inheritance

6.1 Simple inheritance

Objective-C obviously implements the notion of inheritance, but does not support multiple inheritance. This limitation is addressed by other concepts (protocols, class categories) that are explained later in this document (sections 6.4 on the following page, 6.5 on page 22).

C++	Objective-C
<pre>class Foo : public Bar, protected Wiz { }</pre>	<pre>@interface Foo : Bar //single inheritance //An alternative technique must be //used to also "inherit" from Wiz { } @end</pre>

In **C++**, a class can be derived from one or several other classes, using **public**, **protected** or **private** mode. In the methods, one can reference a superclass using the scope resolution operator `::` (`Bar::`, `Wiz::`).

In **Objective-C**, one can derive from one class only, using **public** mode. A method can reference the superclass with the (false) keyword **super**, like in Java.

6.2 Multiple inheritance

Objective-C does not implement multiple inheritance, but brings other concepts, the *protocols* (cf. 6.4 on the following page) and the *categories* (cf. 6.5 on page 22).

6.3 Virtuality

6.3.1 Virtual methods

In Objective-C, all methods are virtual. Hence, the *virtual* keyword does not exist and has no equivalent.

6.3.2 Silent redefinition of virtual methods

It is possible in Objective-C to implement a method that has not been declared in the *interface* section. This feature does not replace the **@private** notion for the methods (even if it can be used to “hide” methods), but does lighten the interface declarations.

This is not a bad practice: the methods one uses with this technique are often “well-known” methods of the super-classes. Many methods of the root-class **NSObject** are silently redefined. One can quote the constructor **init** (cf. section 7.1 on page 24), the destructor **dealloc** (cf. section 7.2 on page 28), the drawing method **drawRect:** of views, and so on.

The interfaces are then lighter and easier to read, even if it is more difficult to see what can be redefined, implying a regular reading of the super class documentation.

The notion of pure virtual methods (a method that must be redefined in sub-classes), is addressed by the concept of *formal protocols* (cf. section 6.4.1 on the following page on the next page).

6.3.3 Virtual inheritance

Virtual inheritance is not relevant in Objective-C, since inheritance cannot be multiple and has none of the associated problems.

6.4 Protocols

Java fixes the lack of multiple inheritance by the notion of *interface*. In Objective-C, the same notion is used, and is called a *protocol*. In C++, this would be an abstract class. A protocol is not a real class: it can only declare methods and cannot hold any data. There are two types of protocols : *formal* and *informal*.

6.4.1 Formal protocol

A formal protocol is a set of methods that must be implemented in any conforming class. This can be seen as a certification regarding a class, ensuring that it is able to handle everything that is necessary for a given service. A class can conform to an unlimited number of protocols.

C++
<pre>class MouseListener { public: virtual bool mousePressed(void) = 0; //pure virtual method virtual bool mouseClicked(void) = 0; //pure virtual method }; class KeyboardListener { public: virtual bool keyPressed(void) = 0; //pure virtual method }; class Foo : public MouseListener, public KeyboardListener {...} //Foo MUST implement mousePressed, mouseClicked and keyPressed //It can then be used as an event listener for the mouse and the keyboard</pre>

Objective-C
<pre>@protocol MouseListener -(BOOL) mousePressed; -(BOOL) mouseClicked; @end @protocol KeyboardListener -(BOOL) keyPressed; @end @interface Foo : NSObject <MouseListener, KeyboardListener> { ... } @end //Foo MUST implement mousePressed, mouseClicked and keyPressed //It can then be used as an event listener for the mouse and the keyboard</pre>

In C++, a protocol is implemented by an abstract class and pure virtual methods. The abstract class in C++ is more powerful than the Objective-C protocol since it can contain data.

In Objective-C, the protocol is a specific concept. The syntax using angular brackets `<...>` is not linked to the C++ templates, a feature that does not exist in Objective-C.

A class can implement all the methods of a protocol without declaring its conformance. In this case, the `conformsToProtocol:` method returns NO. For efficiency reasons, `conformsToProtocol:` does not check, method-by-method, the conformance to a protocol, but is based on the explicit declaration from the developer. However, the negative answer to `conformsToProtocol:` does not prevent the program from behaving correctly if the methods of the protocol are called. Here is the prototype of `conformsToProtocol:`

```
-(BOOL) conformsToProtocol:(Protocol*)protocol
//a Protocol object is returned by a call to @protocol(protocol name)
```

The type of an object that conforms to a formal protocol can be added the name of the protocol itself, between angular braces. This is useful for assertions. For example :

```
//the following standard Cocoa method takes one parameter which is of
//any type (id), but must conform to the protocol NSDraggingInfo
-(NSDragOperation) draggingEntered:(id <NSDraggingInfo>)sender;
```

6.4.2 Informal protocol

It may be desirable that a class conforms to a protocol, to show that it can handle a particular service, but without forcing it to conform to the *whole* protocol. For instance, in Cocoa, the notion of a *delegate* object is widely used: an object can be given an assistant, to handle some tasks, but not all of them.

An immediate solution would be to split a formal protocol into multiple ones, and then make the class conform to a subset of these protocols. This is not very handy. Cocoa brings a solution in the notion of *informal protocols*. Surprisingly, an informal protocol is not declared with the relaxation of a formal protocol. Another concept is used : the *class category* (cf. section 6.5 on the following page).

Let us imagine a service called “document managing”. Assume there is a difference between green, blue and red documents. Even if the class can only handle blue documents, a `Slave` class is preferred to using three formal protocols: `manageGreenDocuments`, `manageBlueDocuments` and `manageRedDocuments`. To the `Slave` class is rather added a *category* `DocumentsManaging`, declaring the methods for the tasks it is able to accomplish. The name of the category is specified in parenthesis (more explanations are given in Section 6.5 on the next page):

```
@interface Slave (DocumentsManaging)
-(void) manageBlueDocuments:(BlueDocument*)document;
-(void) trashBlueDocuments:(BlueDocument*)document;
@end
```

Any class can use the `DocumentsManaging` category, to declare methods that are related to that service.

```
@interface PremiumSlave (DocumentsManaging)
-(void) manageBlueDocuments:(BlueDocument*)document;
-(void) manageRedDocuments:(RedDocument*)document;
@end
```

A developer can then browse the code and see the `DocumentsManaging` category. Hence, he can suppose that the class is useful for some tasks, and he can check which ones exactly by consulting the documentation. Even if he does not check the source code, a run-time request is still possible:

```
if ([mySlave respondsToSelector:@selector(manageBlueDocuments:)]
    [mySlave manageBlueDocuments:document];
```

Strictly speaking, apart from the knowledge of the prototypes, the informal protocol is useless to the compiler, it does not restrict the use of objects. However, it is vital for self-documenting code, making APIs more readable.

6.4.3 Message qualifiers for distant objects

Thanks to the dynamism of Objective-C, distant objects can communicate easily. They can belong to distinct programs, on different machines, but can delegate some tasks and exchange some information. Now, formal protocols are a perfect way to ensure that an object conforms to a given service, wherever it comes from. The formal protocol concept has been given some extra keywords to allow for more efficient communication between distant objects.

These keywords are **in**, **out**, **inout**, **bycopy**, **byref** and **oneway**. It is only applicable to distributed objects, and outside a protocol definition, they are not reserved keywords and can be freely used.

These keywords are inserted inside the prototypes of the methods declared inside a formal protocol, to add extra information regarding their behaviour. They can be used to specify which parameters are *input* parameters, and which are *output* results; it is also possible to tell whether they are to be used by copy or by reference; and the methods can be made synchronous or not. Here are the different meanings:

- an **in** parameter is an input variable;
- an **out** parameter is an output variable;
- an **inout** parameter can be used in both ways (input and output);
- a **bycopy** parameter is transmitted by copy ;
- a **byref** parameter is transmitted by reference (without copy) ;
- a **oneway** method is asynchronous (the result is not immediately expected) - hence it must return **void**.

For instance, this is an asynchronous method that returns an object:

```
-(oneway void) giveMeAnObject:(bycopy out id *)anObject;
```

By default, parameters are considered to be **inout**, except **const** pointers, that are supposed to be **in**. Choosing **in** or **out** instead of **inout** is an optimization. The default mode to transmit the parameters is **byref**, and the methods are synchronous by default (without **oneway**).

For parameters transmitted by value, like non-pointers variables, **out** and **inout** make no sense, only **in** is correct.

6.5 Class categories

Creating *categories* for a class is a way to split its implementation into several parts. Each *category* is a part of the class. A class can use an arbitrary number of categories, but none can add instance data. It brings the following benefits:

- For the meticulous developer, it is possible to make groups of methods. For a very rich class, its different roles can be cleanly separated;
- It is possible to compile them separately, making collaborative work possible on the same class;
- If the interface of a category and its implementation are present in some implementation file (*.m* file), it is very easy to define *private* methods, which are only visible inside the file (even if anybody who knows the prototype can use them, there is no calling restriction). An appropriate name for such a category could be *FooPrivateAPI*;

- A class can be extended differently in different applications, without duplicating the common code. Any class can be extended, even existing Cocoa classes.

The last point is important: each developer would have their preference to extend a standard class with the methods that are useful for them. This is not a real problem: inheritance is a way to do that. However, in the context of simple inheritance, this can lead to a heavy subclassing scheme. Moreover, it can seem a disproportional effort to make a subclass for a single method. The class categories are an elegant solution to this problem.

C++	Objective-C
<pre>class MyString : public string { public: //counts the vowels int vowelCount(void); }; int MyString::vowelCount(void) { ... }</pre>	<pre>@interface NSString (VowelsCounting) //please note that no brackets {} are used -(int) vowelCount; //counts the vowels @end @implementation NSString (VowelsCounting) -(int) vowelCount { ... } @end</pre>

In C++, the new class is usable without restrictions.

In **Objective-C**, the NSString class (a standard Cocoa class) is given an extension that is usable inside the whole program. No new class is created. Every NSString object benefits from the extension (even constant strings, cf. section 10.1 on page 44). But no instance variable can be added inside a category, so there is no `{...}` block.

6.6 Joint use of protocols, categories, subclassing:

The only restriction in the joint specification of protocols, categories and derivation, is that a subclass and a category cannot be declared concurrently; two steps are needed.

```
@interface Foo1 : SuperClass <Protocol1, Protocol2, ... > //ok
@end

@interface Foo2 (Category) <Protocol1, Protocol2, ... > //ok
@end

//below : compilation error
@interface Foo3 (Category) : SuperClass <Protocol1, Protocol2, ... >
@end

//a solution :
@interface Foo3 : SuperClass <Protocol1, Protocol2, ... > //step 1
@end
@interface Foo3 (Category) //step 2
@end
```

7 Instantiation

The instantiation of a class leads to two problems: how are the notions of a constructor/destructor/copy operator implemented, and how are they managed in memory ?

First, an important point: in the C and C++ languages, variables are said to be “automatic” by default: unless they are declared **static**, they only exist inside their definition block. Only dynamically allocated memory is usable beyond, until the matching **free()** or **delete** method is called. Objects follow this rule in C++.

However, in Objective-C, **all objects are dynamically allocated**. This is rather logical, C++ being very static, and Objective-C being very dynamic. Objective-C’s dynamism wouldn’t be that rich if objects weren’t created at run-time.

Please see the section 8 on page 33 for a detailed explanation about the way to retain or release objects.

7.1 Constructors, initializers

7.1.1 Distinction between *allocation* and *initialization*

In C++, allocation and initialization of an object are mixed when calling the constructor. In Objective-C, they are two different methods.

Allocation is handled by the **class method** **alloc**, which also initializes all instance data. Instance data are set to 0, except the **isa** (is-a) pointer of **NSObject**, which value describes the exact type of the newly created object at run-time. Should instance data be set to a particular value, depending on the parameters of construction, then the corresponding code is deported inside an **instance method**. Its name usually begins with *init*. Therefore, the construction is clearly divided into two steps : allocation and initialization. The **alloc** message is sent to the *class*, and the **init...** message is sent to the *object* newly instantiated by **alloc**.

The initialization step is not optional, and alloc should always be followed by init; with successive calls to superclass initializers, this must end in calling the **init** method of **NSObject**, which performs important work.

In C++, the name of the constructor cannot be chosen. In Objective-C, the initializer is a method like any other, and the **init** prefix is traditional but not mandatory. However, you are strongly encouraged to follow a rule: **the name of an initialization method must begin with “init”**.

7.1.2 Using **alloc** and **init**

A call to **alloc** returns a new object that must be sent **init** . The call to **init** also returns an object. Most of the time, this will be the initial object. Sometimes, when one is using a singleton (an object for which a single instance is allowed), **init** could substitute another return value. So, the returned value of **init** should not be ignored. Usually, **alloc** and **init** are called on the same line.

C++
<pre>Foo* foo = new Foo;</pre>
Objective-C
<pre>Foo* foo1 = [Foo alloc]; [foo1 init]; //bad : the returned value should be used Foo* foo2 = [Foo alloc]; foo2 = [foo2 init]; //ok, but not handy Foo* foo3 = [[Foo alloc] init]; //ok, usual way to do</pre>

To know whether an object was created or not, C++ requires either exception catching, or a test against 0 (if **new(nothrow)** was used). With Objective-C, a test against **nil** is sufficient.

7.1.3 Example of a correct initializer

The constraints for a correct initializer are

- its name begins with *init*;
- it returns the object to use;
- it calls some `init` method of the superclass, so that `init` of `NSObject` will finally be called;

Here is an example of object instantiation, in C++ and in Objective-C.

C++
<pre>class Point2D { public: Point2D(int x, int y); private: int x; int y; }; Point2D::Point2D(int anX, int anY) {x = anX; y = anY;} ... Point2D p1(3,4); Point2D* p2 = new Point2D(5, 6);</pre>
Objective-C
<pre>@interface Point2D : NSObject { int x; int y; } //Note : id is somewhat similar to void* in Objective-C. //(id) is the most "general" type for an object -(id) initWithX:(int)anX andY:(int)anY; @implementation Point2D -(id) initWithX:(int)anX andY:(int)anY { //an initializer of the superclass must be called if (![super init]) //if the superclass is NSObject, this must be init return nil; //in case of failure, return nil //in case of success, make additional initializations x = anX; y = anY; return self; //and return the object itself } @end ... Point2D* p1 = [[Point2D alloc] initWithX:3 andY:4];</pre>

Polemic: Most people use the instruction `self = [super init]; if (self) {...}` in the initializer. This is justified by the fact that in some cases, `[super init]` could return a different object. However, in a very interesting document [6], Will Shipley shows that is a bad practice. It is far more logical to use the above form, where it is only checked whether `[super init]` returns `nil` or not.

The successive application of `alloc` and `init` can sound laborious in some cases. Fortunately, it can be shortened a *convenience constructor*. The exact knowledge of such a constructor relies on notions of memory management in Objective-C. So, the exact explanations are given further in this document in Section 8.4.6 on page 36. Briefly, such a constructor, the name of which should be prefixed by the name of the class, behaves like an `init` method, but it performs the `alloc` itself. However, the object is registered in the auto-release pool (cf. section 8.4 on page 33) and will have a limited life cycle if it is not sent a `retain` message. Here is an example :

```
//laborious
NSNumber* tmp1 = [[NSNumber alloc] initWithFloat:0.0f];
...
[tmp1 release];

//handier
NSNumber* tmp2 = [NSNumber numberWithFloat:0.0f];
...
//no need to release
```

7.1.4 Initialization failure

We have seen that the initializer starts by calling an initializer of the superclass. If this call fails, the `nil` value is returned. But in this case, what would happen to the current object? By sending `nil` instead of `self`, is there a memory leak ? Should we not ask the object to release, since `alloc` has allocated some memory ?

In fact, this is automatic. If the superclass is well-implemented, it will trigger a `dealloc` (see Section 7.2 on page 28) in case of failure. But if you decide to give up your part of the initialization and cancel the object creation (returning `nil`), then you first must deallocate the object with `[self autorelease]` yourself (memory management is explained in Section 8 on page 33).

7.1.5 Default constructor : designated initializer

The notion of default constructor has no real meaning in Objective-C. Because all objects are allocated dynamically, their construction is always explicit. However, a preferred constructor can be used to factorize some code. Indeed, a correct initializer is most of the time similar to:

```
if (![super init]) // "init" or another appropriated
    return nil;    // initializer of the superclass

// in case of success, add some code...
return self;
```

Since code redundancy is a bad practice, it seems heavy-handed to repeat that scheme in each possible initializer. The best solution is to polish up the most essential initializer with this code. Then the other initializers will call this “preferred” initializer, known as the *designated initializer*. Logically, the designated initializer is the one with the greatest number of parameters, since it is not possible in Objective-C to give default values to these parameters.

```

-(id) initWithX:(int)x
{
    return [self initWithX:x andY:0 andZ:0];
}

-(id) initWithX:(int)x andY:(int)y
{
    return [self initWithX:x andY:y andZ:0];
}

//designated initializer
-(id) initWithX:(int)x andY:(int)y andZ:(int)z
{
    if (![super init])
        return nil;
    self->x = x;
    self->y = y;
    self->z = z;
    return self;
}

```

If the designated initializer is not the one with the greatest number of parameters, then it's not very useful:

```

//the following code is not handy
-(id) initWithX:(int)x //designated initializer
{
    if (![super init])
        return nil;
    self->x = x;
    return self;
}

-(id) initWithX:(int)x andY:(int)y
{
    if (![self initWithX:x])
        return nil;
    self->y = y;
    return self;
}

-(id) initWithX:(int)x andY:(int)y andZ:(int)z
{
    if (![self initWithX:x])
        return nil;
    self->y = y;
    self->z = z;
    return self;
}

```

7.1.6 List of initialization and default value of instance data

The idea of *list of initialization* for C++ constructors does not exist in Objective-C. However, it is remarkable that, unlike the C++ behaviour, the `alloc` method of Objective-C initializes all the bits of instance data to 0, so the pointers are set to `nil`. This would be a problem in C++ for objects as attributes, but in Objective-C, the objects are always represented by pointers.

7.1.7 Virtual constructor

It is possible in Objective-C to get real virtual constructors. For more details, please see Section 8.4.6 on page 36, after the introduction to the memory management (Section 8 on page 33).

7.1.8 Class constructors

In Objective-C, since the classes are themselves objects, they supply a constructor that can be redefined. It is obviously a class method, inherited from `NSObject`; its prototype is `+(void) initialize;`

This method is automatically called when using the class or one of its sub-classes for the first time. However, it is not true that this method is called only once for a given class; indeed, if a sub-class does not redefine `+(void) initialize`, the Objective-C mechanism calls `+(void) initialize` from the mother class.

7.2 Destructors

In C++, the destructor, like the constructor, is a specific method that can be redefined. In Objective-C, it is an instance method named `dealloc`.

In C++, the destructor is called automatically when an object is freed; it is the same with Objective-C; only the way to release the object is different (cf. Section 8 on page 33).

The destructor should never be called explicitly. In fact, in C++ there is one case where it can be : when the developer himself is managing the memory pool used for the allocation. But in Objective-C, no case justifies an explicit call to `dealloc`. One can use custom memory zones in Cocoa, but their use does not influence common allocation/deallocation practices (cf. Section 7.3 on the next page).

C++
<pre>class Point2D { public: ~Point2D(); }; Point2D::~~Point2D() {}</pre>
Objective-C
<pre>@interface Point2D : NSObject //-(void) dealloc; // in Objective-C, such a "private" method does not // need to be declared in the interface @end @implementation Point2D //in this example the redefinition was not necessary -(void) dealloc { [super dealloc]; //do not forget to transmit to the superclass } @end</pre>

7.3 Copy operators

7.3.1 Classical cloning, `copy` and `copyWithZone:`

In C++, it is important to define a coherent implementation of the copy constructor and the affectation operator. In Objective-C, operator overloading is impossible; one must only ensure that the cloning method is correct.

Cloning in Cocoa is associated with a protocol (cf. Section 6.4 on page 20) named `NSCopying`, requesting the implementation of the method

```
-(id) copyWithZone:(NSZone*)zone;
```

Its argument is a memory zone in which the clone should be allocated. Cocoa allows the use of different custom zones: some methods take such a zone as an argument. Most of the times, the default zone is perfect, and there is no need to specify it each time. Fortunately, `NSObject` supplies a method

```
-(id) copy;
```

which encapsulates a call to `copyWithZone:` with the default zone as parameter. But it is the `copyWithZone:` version which is required by `NSCopying`. Finally, the utility function `NSCopyObject()` provides a slightly different approach, that can be simpler but also requires caution. First, the code is presented without taking `NSCopyObject(...)` into account, for which explanations are given in section 7.3.2 on the next page.

The implementation of `copyWithZone:` for some `Foo` class would look like:

```
//if the superclass does not implement copyWithZone:, and that NSCopyObject()
//is not used
-(id) copyWithZone:(NSZone*)zone
{
    //a new object must be created
    Foo* clone = [[Foo allocWithZone:zone] init];
    //instance data must be manually copied
    clone->integer = self->integer; // "integer" is here of type "int"
    //trigger the same mechanism for sub-objects to clone
    clone->objectToClone = [self->objectToClone copyWithZone:zone];
    //some sub-objects may not be cloned but shared
    clone->objectToShare = [self->objectToShare retain]; //cf. memory management
    //if there is a mutator, it can be used in both cases
    [clone setObject:self->object];
    return clone;
}
```

Note the use of `allocWithZone:`, instead of `alloc`, to handle the zone parameter. `alloc` encapsulates a call to `allocWithZone:` with the default zone. To learn more about Cocoa's custom zone management, refer to the Cocoa documentation.

However, one must take care of the possible implementation of `copyWithZone:` in the superclass.

```

//if the superclass implements copyWithZone:, and that NSCopyObject is not used
-(id) copyWithZone:(NSZone*)zone
{
    Foo* clone = [super copyWithZone:zone]; //creates the new object
    //you must clone instance data specific to the current sub-class
    clone->integer = self->integer; // "integer" is here of type "int"
    //trigger the same mechanism for sub-objects to clone
    clone->objectToClone = [self->objectToClone copyWithZone:zone];
    //some sub-objects may not be cloned but shared
    clone->objectToShare = [self->objectToShare retain]; //cf. memory management
    //if there is a mutator, it can be used in both cases
    [clone setObject:self->object];
    return clone;
}

```

7.3.2 NSCopyObject()

The NSObject class does not implement the NSCopying protocol, that is why a direct sub-class cannot benefit from a call to [super copy...], and must use a standard initialization based on the [... alloc] init] scheme.

The utility function NSCopyObject() can be used to write simpler code, but requires some caution with pointer members (including objects). This function creates a binary copy of an object and its prototype is :

```

//the extra bytes is usually 0, but can be used for the extra space of
//indexed instance data
id <NSObject> NSCopyObject(id <NSObject> anObject,
                           unsigned int extraBytes, NSZone *zone)

```

The binary copy can automate the copy of instance data that are non-pointers; but for a pointer member (including objects), one must keep in mind that it silently creates an additional reference to the pointed data. The usual practice is to reset the pointers afterwards with values coming from correct clonings.

```

//if the superclass does not implement copyWithZone:
-(id) copyWithZone:(NSZone*)zone
{
    Foo* clone = NSCopyObject(self, 0, zone); //binary copy of data
    //clone->integer = self->integer; //useless : binary data already copied

    //a sub-object to clone must be really cloned
    clone->objectToClone = [self->objectToClone copyWithZone:zone];
    //a sub-object to share must only register the new reference
    [clone->objectToShare retain]; //cf. memory management

    //The mutator is likely to release clone->object. This is undesirable,
    //because of the binary copy of the pointer value
    //Therefore, before using the mutator, the pointer is reset
    clone->object = nil;
    [clone setObject:self->object];
    return clone;
}

```

```

//if the superclass implements copyWithZone:
-(id) copyWithZone:(NSZone*)zone
{
    Foo* clone = [super copyWithZone:zone];
    //does a superclass implement NSCopyObject() ? This is important
    //to know what remains to be done

    clone->integer = self->integer;//only if NSCopyObject() has not been used
    //In case of doubt, it can be done systematically

    //a sub-object to clone must still be really cloned
    clone->objectToClone = [self->objectToClone copyWithZone:zone];

    //NSCopyObject() or not, a retain must be done (cf. memory management)
    clone->objectToShare = [self->objectToShare retain];

    clone->object = nil; //in case of doubt, it's better to reset
    [clone setObject:self->object];

    return clone;
}

```

7.3.3 Dummy-cloning, mutability, mutableCopy and mutableCopyWithZone:

When cloning an object that cannot change, a fundamental optimization is to pretend that it is cloned; instead of duplicating it, a reference to it can be returned. Starting from that, we can distinguish the notion of an *immutable* and *mutable* object.

An immutable object cannot have any of its instance data changed; only the initializer gives it a valid state. In this case, it can safely be “pseudo-cloned” by returning only a reference to itself. Since neither it nor its clone can be modified, none of them can be undesirably affected by a modification of the other. A very efficient `copyWithZone:` implementation can be proposed in that case:

```

-(id) copyWithZone:(NSZone*)zone
{
    //the object returns itself, counting one more reference
    return [self retain]; //see the section about memory management
}

```

The use of `retain` comes from the memory management in Objective-C (cf. Section 8 on page 33). The reference counter is incremented by 1 to make the existence of the clone “official” so that deleting that clone will not destroy the original object.

“Pseudo-cloning” is not a marginal optimization. Creating an object requests a memory allocation, which is a “long” process that should be avoided when possible. That is why it is interesting to identify two kind of objects: immutable ones, for which cloning can be fictive, and the others. Making the distinction is as simple as creating “immutable” classes and optionally subclassing them into “mutable” versions, adding methods to change their data instance. For instance, in Cocoa, `NSMutableString` is a subclass of `NSString`, `NSMutableArray` is a subclass of `NSArray`, `NSMutableData` is a subclass of `NSData`, and so on.

However, with the techniques presented here, it seems impossible to get a real clone, safely mutable, from an immutable object which would only know how to “pseudo-clone” itself. Such a limitation would significantly decrease the usefulness of immutable objects, isolating them from the “external world”.

In addition to the `NSCopying` protocol, there is another protocol (cf. Section 6.4 on page 20) named `NSMutableCopying`, requesting the implementation of

```
-(id) mutableCopyWithZone:(NSZone*)zone;
```

The `mutableCopyWithZone:` method must return a mutable clone, where modifications would not apply to the original object. Similarly to the `copy` method, there is a `mutableCopy` method which automatically calls `mutableCopyWithZone:` by giving it the default zone. The implementation of `mutableCopyWithZone:` looks like the following, similar to the classical copy previously presented:

```
//if the superclass does not implement mutableCopyWithZone:
-(id) mutableCopyWithZone:(NSZone*)zone
{
    Foo* clone = [[Foo allocWithZone:zone] init]; //or NSCopyObject(...)
    clone->integer = self->integer;
    //Like with copyWithZone:, some sub-objects can be cloned, some others shared
    //A mutable sub-object can be cloned with a call to mutableCopyWithZone:
    //...
    return clone;
}
```

Do not forget to use the possible `mutableCopyWithZone:` of the superclass:

```
//if the superclass implements mutableCopyWithZone:
-(id) mutableCopyWithZone:(NSZone*)zone
{
    Foo* clone = [super mutableCopyWithZone:zone];
    //...
    return clone;
}
```


8 Memory management

8.1 new and delete

The C++ keywords `new` and `delete` do not exist in Objective-C (`new` exists as a method, but it is just a deprecated shortcut for `alloc+init`). They are respectively replaced by calls to `alloc` (cf. Section 7.1 on page 24) and `release` (cf. Section 8.2 on this page).

8.2 Reference counting

Memory management in Objective-C is one of the most important parts of the language. In C or C++, a memory area is allocated once and freed once. It can be referenced with as many pointers as desired, but only one pointer will be given the `delete` call.

On the other hand, Objective-C implements a reference counting scheme. An object knows how many times it is referenced. This can be explained by the analogy of dogs and leashes (an analogy directly taken from *Cocoa Programming for MacOS X* [4]). If an object is a dog, everyone can ask for a leash to hold it. If someone does not care about the dog anymore, it can drop its leash. While the dog has at least one leash, it must stay there. But as soon as the number of leashes falls to 0, the dog is free!

More technically, the reference counter of a newly created object is set to 1. If a part of the code needs to reference that object, it can send it a `retain` message, which will increase the counter by one. When a part of the code does not need the object any more, it can send it a `release` message, that will decrease the counter by 1.

An object can receive as many `retain` and `release` messages as needed, as long as the reference counter has a positive value. As soon as it falls to 0, the destructor `dealloc` is automatically called. Sending `release` again to the address of the object, which is now invalid, triggers a memory fault.

This technique is not equivalent to the `auto_ptr` from the C++ STL. On the contrary, the Boost library [2] supplies an encapsulation of pointers into a `shared_ptr` class, which implements the reference counting scheme. But it is not part of the standard library.

8.3 alloc, copy, mutableCopy, retain, release

The understanding of memory management does not fully explain how it is used. The goal of this section is to give some rules. The keyword `autorelease` is left aside for now, since it is more difficult to understand.

The basic rule to apply is **Everything that increases the reference counter with `alloc`, `[mutable]copy[WithZone:]` or `retain` is in charge of the corresponding `[auto]release`.** Indeed, these are the three ways to increment the reference counter. It also means that you should take care of releasing an object in only a limited number of cases:

- when you explicitly instantiate an object with `alloc`;
- when you explicitly clone the object with `copy[WithZone:]` or `mutableCopy[WithZone:]` (whatever the copy is : a real or a pseudo-clone);
- when you explicitly use `retain`.

The field of application of `retain` is rather limited. But limited does not mean that it is rare. In the next section, the concept of `autorelease` shows a common use of `retain`.

Please remember that by default, it is legal to send a message (like `release`) to `nil`, without any consequence (see Section 5.4.1 on page 17).

8.4 autorelease

8.4.1 autorelease

The rule stated in the previous section is so important that it bears repeating: **Everything that increases the reference counter with `alloc`, `[mutable]copy[WithZone:]` or `retain` is in charge of the corresponding `[auto]release`.**

In fact, with the mere `alloc`, `retain` and `release`, this rule could not be applied. Indeed, there are some methods that are not constructors, but that are designed to create objects : for instance a binary addition operator in C++ (`obj3 operator+(obj1, obj2)`). In C++, the returned object would be allocated on the stack and automatically destroyed when leaving its scope. But in Objective-C, such objects do not exist. The function has necessarily used `alloc`, but cannot release the object before returning it on the stack ! Here are given some illustrations of what is going wrong:

```
-(Point2D*) add:(Point2D*)p1 and:(Point2D*)p2
{
    Point2D* result = [[Point2D alloc] initWithX:([p1 getX] + [p2 getX])
                                                andY:([p1 getY] + [p2 getY])];

    return result;
}
//ERROR : the function performs "alloc", so, it is creating
//an object with a reference counter of 1. According
//to the rule, it should destroy the object.

//This can lead to a memory leak when summing three points :
[calculator add:[calculator add:p1 and:p2] and:p3];

//The result of the first addition is anonymous
//and nobody can release it. It is a memory leak.
```

```
-(Point2D*) add:(Point2D*)p1 and:(Point2D*)p2
{
    return [[Point2D alloc] initWithX:([p1 getX] + [p2 getX])
                                      andY:([p1 getY] + [p2 getY])];
}
//ERROR : This is exactly the same code as above. The fact that
//no intermediate variable is used does not change anything.
```

```
-(Point2D*) add:(Point2D*)p1 and:(Point2D*)p2
{
    Point2D* result = [[Point2D alloc] initWithX:([p1 getX] + [p2 getX])
                                                andY:([p1 getY] + [p2 getY])];

    [result release];
    return result;
}
//ERROR : obviously, it is nonsense to destroy the object after creating it
```

The problem seems intractable. It would be, if `autorelease` was not there. To simplify, let us say that sending `autorelease` to an object means that it is sent a `release` that will be performed “later”. But “later” does not mean “at any time”; this is detailed in Section 8.4.2 on the following page. For a first approach, here is the only possible solution:

```
-(Point2D*) add:(Point2D*)p1 and:(Point2D*)p2
{
    Point2D* result = [[Point2D alloc] initWithX:([p1 getX] + [p2 getX])
                                                andY:([p1 getY] + [p2 getY])];

    [result autorelease];
    return result;          //a shorter writing is "return [result autorelease]"
}
//CORRECT : "result" will be automatically released later,
//after being used in the calling code
```

8.4.2 The autorelease pool

In the previous section, **autorelease** has been presented as a kind of magical **release** that is automatically applied at the right moment. But it would make no sense to let the compiler guess what the right moment is. In this case, a garbage collector would be more useful. To explain how it works, more details must be given about **autorelease**.

Each time an object receives **autorelease**, it is only registered into an “autorelease pool”. When the pool is destroyed, the object receives a real **release**. The problem has moved : how is this pool handled ?

There is not a single answer : if you use Cocoa for an application with a graphical interface, most of the time there is nothing to do. Otherwise, you would have to create and destroy the pool yourself.

An application with a graphical interface generally uses an *event loop*. Such a loop waits for actions from the user, then wakes up the program to perform the action, then goes back to sleep until the next event. When you create a graphical application with Cocoa, an autorelease pool is automatically created at the beginning of the loop and destroyed at the end. This is logical: generally, a user’s action triggers a cascade of tasks. Temporary objects are created, then destroyed, since they do not have to be kept for the next event. If some of them must be persistent, the developer must use **retain** as necessary.

On the other hand, when there is no graphical interface, you have to create an autorelease pool around the code that is needing it. When an object receives **autorelease**, it knows how to find the closest autorelease pool. Then, when it is time to empty the pool, you can just destroy it with a simple **release**. Typically, a command-line Cocoa program contains the following code:

```
int main(int argc, char* argv[])
{
    NSAutoreleasePool* pool = [[NSAutoreleasePool alloc] init];
    //...
    [pool release];
    return 0;
}
```

8.4.3 Using several autorelease pools

It is possible, and sometimes useful, to have more than one autorelease pool in the program. An object that receives **autorelease** will register in the closest pool. Therefore, if a function creates and uses a large number of temporary objects, increased performance can be obtained by creating a local autorelease pool. That way, the crowd of temporary objects will be destroyed as soon as possible and will not clutter the memory after the function has returned. Even if it should be known, this technique is only used in particular circumstances where memory footprint is a real problem.

8.4.4 Caution with autorelease

The fact that **autorelease** is handy should not lead to its misuse.

- First, sending more **autorelease** calls than necessary is similar to sending too many **release** calls: it triggers a memory fault when emptying the pool;
- Then, even if it is true that any **release** message can be replaced by **autorelease**, this would be a performance hit, since the autorelease pool is more work than a normal **release**. Moreover, delaying all deallocations can lead to useless and irrelevant peaks of memory consumption.

8.4.5 autorelease and retain

Thanks to **autorelease**, a method can create an object that can plan its own releasing. However, it is common that the object must be kept longer. In this case, one has to send a **retain** on it, and

plan for a release. This is the most common use of **retain**, typically illustrated with convenience constructors (cf. Section 8.4.6 on this page) or setters (cf. section 8.4.7 on the next page).

8.4.6 Convenience constructor, virtual constructor

The successive applications of **alloc** and **init** can sound laborious in some cases. Fortunately, it can be shortened by the notion of *convenience constructor*. Such a constructor, the name of which should be prefixed by the name of the class, behaves like an **init** method, but it performs the **alloc** itself. However, the returned object is registered inside an autorelease pool, and will be temporary if it is not sent any **retain**. For example:

```
//laborious
NSNumber* zero_a = [[NSNumber alloc] initWithFloat:0.0f];
...
[zero_a release];

...

//handier
NSNumber* zero_b = [NSNumber numberWithFloat:0.0f];
...
//no need of release
```

With help from the section about memory management (Section 8 on page 33), it's obvious that such a constructor relies on **autorelease**. The underlying code is not that obvious anyway, since it requires the correct use of **self**. Indeed, a convenience constructor is a *class method*, so that **self** refers to an object of type **Class**, which is a *meta-class* instance. In an *initializer*, which is an *instance* method, **self** is an *instance* of the class, and refers to a “normal” object.

It is easy to write a bad convenience constructor. Let us assume a class **Vehicle** holding a color, and supplying a convenience constructor.

```
//The Vehicle class
@interface Vehicle : NSObject
{
    NSColor* color;
}
-(void) setColor:(NSColor*)color;

//convenience constructor
+(id) vehicleWithColor:(NSColor*)color;
@end
```

The implementation of the convenience constructor is somewhat subtle.

```
//bad convenience constructor
+(Vehicle*) vehicleWithColor:(NSColor*)color
{
    //the value of "self" should not change
    self = [[self alloc] init]; // ERROR !
    [self setColor:color];
    return [self autorelease];
}
```

self in this class method refers to the *class*. It cannot be set to be the *instance*.

```
//Almost perfect constructor
+(id) vehicleWithColor:(NSColor*)color
{
    id newInstance = [[Vehicle alloc] init]; // OK, but ignores potential
                                              // sub-classes

    [newInstance setColor:color];
    return [newInstance autorelease];
}
```

We can still improve this. In Objective-C one can get the behaviour of a virtual constructor. The constructor just needs to perform an introspection to know which is the real class of the object performing the method. Then it can directly produce an object of the right subclass. The false keyword `class` can be used; this is a method of `NSObject` that returns the current object's class object (a meta-class instance).

```
@implementation Vehicle
+(id) vehicleWithColor:(NSColor*)color
{
    id newInstance = [[[self class] alloc] init]; // PERFECT, the class is
                                                  // dynamically identified

    [newInstance setColor:color];
    return [newInstance autorelease];
}
@end

@interface Car : Vehicle {...}
@end

...

//produces a (red) car
id car = [Car vehicleWithColor:[NSColor redColor]];
```

Similar to the rule about the **init** prefix for initializers, you are strongly encouraged to prefix a convenience constructor with the name of the class. There are very few cases where this is not the case, like `[NSColor redColor]` in the previous code, which should have been written `[NSColor colorRed]`.

Finally, let us repeat the rule: **Everything that increases the reference counter with `alloc`, `[mutable]copy` `[WithZone:]` or `retain` is in charge of the corresponding `[auto]release`.** When calling a convenience constructor, you are not explicitly calling `alloc`, so you are not in charge of the `release`. However, when creating such a constructor, you are writing `alloc`, and you must not forget the `autorelease`.

8.4.7 Setter

A setter (mutating accessor) is a typical exemple of something that is difficult to write without the knowledge of memory management in Objective-C. Let us assume a class encapsulating an `NSString` named *title*, and let us suppose that we want to change the value of this string. This very simple example raises all the problems related to setters. It is important to define a coding *style*, that must be clever. Inside a setter, the parameter is an object that can have different meanings:

- If it is a pointer towards an external object, it should not be copied, but only retained, so that any external modification will be visible inside the current encapsulating object being changed (case #1);
- if the intention is to become a copy of an external object, it must be cloned (case #2);

- in case #2, if the parameter object is a temporary object that was created on the fly for that call, retaining it instead of copying it is an excellent optimization. Thanks to the reference counter, it can be “pseudo-destroyed” in the external world with **release**, even if it is still living inside the encapsulating object (case #3).

In C++, the optimization of case #3 cannot be applied; but for cases #1 and #2, the difference can be automatically detected according to the fact that the parameter is a pointer or an automatic object (not allocated by **new**). This is not possible in Objective-C, where all objects are dynamically created : only pointers are handled. That constraint leads to a difficult choice at first glance: who is in charge of the copy (case #2) or non-copy (case #1) of the parameter object ? Is it the developer of the setter, or its user ? If it is the user, he must have read the documentation of the setter and be aware of what he is doing. If it is the developer, the case #3 optimization cannot be done, since inside the setter, it is not possible to know whether the parameter object is temporary or not.

A common choice is to trust the user. It is reasonable to believe that he has read the documentation (none shall ignore it), rather than drop an optimization. **The setter does not usually copy the parameter, but only makes a retain on it.** Starting from that rule, it is possible to decide how to write a setter, but one must still take care of three common errors:

```
-(void) setTitle:(NSString*)newTitle
{
    title = [newTitle retain];
    //ERROR : memory leak : the old title is no longer referenced
}
```

```
-(void) setTitle:(NSString*)newTitle
{
    [title release];
    title = [newTitle retain];
    //ERROR : if newTitle == title, (it may happen),
    //and that the reference counter of newTitle is 1
    //then it is invalid to use newTitle (title) after [title release]
}
```

```
-(void) setTitle:(NSString*)newTitle
{
    if (title != newTitle)
        [title release]; //ok : release can be sent to nil
    title = [newTitle retain]; //ERROR : should be inside the "if",
    //if title == newTitle, its reference counter should not be increased;
}
```

Let's now compare these three errors to three right ways to do it. These examples are borrowed from *Cocoa Programming for MacOS X* [4].

```
//Technique "Check before you modify"
//the most intuitive way for a C++ developer
-(void) setTitle:(NSString*)newTitle
{
    if (title != newTitle) //avoid degenerated case
    {
        [title release]; //release the old one
        title = [newTitle retain]; //retain the new one
    }
}
```

```
//Technique "Autorelease the old value"
-(void) setTitle:(NSString*)newTitle
{
    [title autorelease]; //correct even if title == newTitle, since the
                        //release occurs later...
    title = [newTitle retain];
    //... which means that this retain occurs before the release
}
```

```
//Technique "retain before you release"
-(void) setTitle:(NSString*)newTitle
{
    [newTitle retain]; //increase the counter by one...
    [title release];  //...so that it cannot fall to 0 here
    title = newTitle; //no retain here ! It has already been done
}
```

Here's one last trick: an NSString can hide an NSMutableString. More generally, a mother-class pointer of an immutable class can hide a mutable subclass. Thus, setting an NSString does not mean that this parameter is immutable in the outside world. The object that receives the string should copy it instead of retain it, if it wants to take a snapshot of the current string state. Otherwise, external modification will be visible inside the set object. Logically, the user should perform the copy himself before calling the setter. However, the pseudo-cloning scheme (cf. section 7.3.3 on page 31) allows the systematic use of `copy` for many classes.

8.4.8 Getter

Since objects are always dynamically allocated in Objective-C, they are all referenced as pointers. Typically, the getters for a value as object will only return the address of the object; these objects are not copied on the fly. Please note that while the name of a getter is usually exactly the same as the data itself, in Objective-C there is no clash of names. For a boolean value, it can be prefixed by `is`.

```

@interface Button
{
    NSString* label;
    BOOL      pressed;
}
-(NSString*) label;
-(void) setLabel:(NSString*)newLabel;
-(BOOL) isPressed;
@end

@implementation Button
-(NSString*) label
{
    return label;
}

-(BOOL) isPressed
{
    return pressed;
}

-(void) setLabel:(NSString*)newLabel
{...}
@end

```

When returning pointers to instance data, a problem is that it is easy for a well-informed user to modify that data, if it is mutable object. It may be desirable to protect the data from external modifications.

```

@interface Button
{
    NSMutableString* label;
}
-(NSString*) label;
@end

@implementation Button
-(NSString*) label
{
    return label; //OK, but the power user will be able to downcast
                //the result to an NSMutableString, and thus modify the string
}

-(NSString*) label
{
    return [NSString stringWithString:label];
    //OK : a new, immutable string, is returned
}
@end

```

8.5 Retain cycles

A retain cycle is something that must be avoided. If an object A retains an object B , and that B and C are mutually retaining each other, B and C are forming a retain cycle.

$$A \rightarrow B \rightleftarrows C$$

If *A* releases *B*, *B* will not be deallocated since it is still retained by *C*. And *C* cannot be deallocated since it is retained by *B*. Retain cycles usually lead to memory leaks.

This is why, in a tree structure, a node usually retains its children, but a child usually does not retain its parent.

8.6 Garbage collector

Objective-C 2.0 (cf. Section 15 on page 50) implements a garbage collector. The use of that garbage collector is optional : you can decide if you want to precisely control the life-time of objects, or if you want to make less bug-prone code. The garbage collector is enabled or disabled for the whole program.

If the garbage collector is enabled, **retain**, **release** and **autorelease** are redefined to do nothing. Thus, code that was written without garbage collection can be recompiled easily for the garbage collector.

9 Exceptions

Exception handling in Objective-C is closer to Java than to C++, particularly because of the `@finally` keyword. `finally` is known in Java but not in C++. It is an additional (but optional) part of a `try()...catch()` block, containing code that will always be executed, whether an exception is caught or not. This is useful to write short and clean code that properly frees resources.

Apart from that, the behaviour of `@try...@catch...@finally` in Objective-C is very classical; however, only objects can be thrown (unlike C++). A simple example with and without `@finally` is shown.

Without finally	With finally
<pre>BOOL problem = YES; @try{ dangerousAction(); problem = NO; } @catch (MyException* e){ doSomething(); cleanup(); } @catch (NSException* e){ doSomethingElse(); cleanup(); //here is the exception re-thrown @throw } if (!problem) cleanup();</pre>	<pre>@try{ dangerousAction(); } @catch (MyException* e){ doSomething(); } @catch (NSException* e){ doSomethingElse(); @throw //here is the exception re-thrown } @finally{ cleanup(); }</pre>

Strictly speaking, `@finally` is not essential, but is a useful tool for handling exceptions properly. As shown in the previous example, it is also handling well the case of a re-thrown exception in a `@catch`. In fact, the `@finally` is triggered when quitting the `@try` scope. An illustration is given below.

```

int f(void)
{
    printf("f: 1-you see me\n");
    //See the section about strings to understand the "@" syntax
    @throw [NSException exceptionWithName:@"panic"
                                         reason:@"you don't really want to known"
                                         userInfo:nil];
    printf("f: 2-you never see me\n");
}

int g(void)
{
    printf("g: 1-you see me\n");
    @try {
        f();
        printf("g: 2-you do not see me (in this example)\n");
    }
    @catch(NSException* e) {
        printf("g: 3-you see me\n");
        @throw;
        printf("g: 4-you never see me\n");
    }
    @finally {
        printf("g: 5-you see me\n");
    }
    printf("g: 6-you do not see me (in this exemple)\n");
}

```

A last point: the `catch(...)` of C++ (with suspension points), that can catch anything, does not exist in Objective-C. Indeed, since only objects can be thrown, it is always possible to catch them with the type id (cf. Section 5.1 on page 9).

Please note that an `NSException` class exists in Cocoa, and that you are encouraged to use it as mother class of what you throw. Thus, a `catch(NSException* e)` should be equivalent to `catch(...)`.

10 Strings in Objective-C

10.1 The only static objects in Objective-C

In the C language, strings are arrays of characters, or `char*` pointers. Handling such data is difficult and causes many bugs. The `string` class of C++ is a relief. In Objective-C, it has been explained in Section 7 on page 24 that objects cannot be automatic and must be allocated at run-time. This is rather incompatible with the use of static strings. This could lead to a situation where static C strings are used as construction parameters of `NSString` objects; this would be unhandy and a waste of memory.

Fortunately, static Objective-C strings do exist. These are simple C strings between quotation marks, with an additional `@` prefix.

```
NSString* notHandy = [[NSString alloc] initWithUTF8String:"helloWorld"];
NSString* stillNotHandy = //initWithFormat is a kind of sprintf()
    [[NSString alloc] initWithFormat:@"%s", "helloWorld"];
NSString* handy = @"hello world";
```

Moreover, a static string can be sent methods like a regular object.

```
int size = [@"hello" length];
NSString* uppercaseHello = [@"hello" uppercaseString];
```

10.2 NSString and encodings

`NSString` objects are very useful, since in addition to a large number of handy methods, they also support different encodings (ASCII, Unicode, ISO Latin 1...). Translation and localization of applications using such strings is really simple.

10.3 Description of an object, %@ format extension, NSString to C string

In Java, every object inherits from `Object` and benefits from a `toString` method, that can describe the object as a string, which is useful for debugging. In Objective-C, this method is named `description` and returns an `NSString`.

The `printf` function of the C language has not been extended to support `NSString` objects. `NSLog` can be used instead, or any other function that accepts a formatted-string like `printf`. For an `NSString` object, the format to use is not “%s” but “%@”. And more generally, “%@” can be used on any object because it is in fact using the returned value of a call to `-(NSString*) description`.

Moreover, an `NSString` can be converted into a C-like string with the `UTF8String` method (previously `cString`).

```
char* name = "Spot";
NSString* action1 = @"running";
printf("My name is %s, I like %s, and %s...\n",
    name, [action1 UTF8String], [@"running again" UTF8String]);
NSLog(@"My name is %s, I like %@ and %@\n",
    name, action1, @"running again");
```

11 STL and Cocoa

The standard C++ library is one of its strengths. Even if it has some gaps (for instance in the functors; gaps usually filled with the SGI STL [5]), it is very complete. This is not a real part of the language, since it is only an extension, but it is common to find such equivalents in other languages. In Objective-C, you have to look at Cocoa to find containers, iterators and other ready-to-use algorithms.

11.1 Containers

Obviously, Cocoa does things in the object-oriented way: a container is not a template, it can only contain objects. At the time of writing, the available containers are:

- `NSArray` and `NSMutableArray` for ordered collections;
- `NSSet` and `NSMutableSet` for unordered collections;
- `NSDictionary` and `NSMutableDictionary` for key/value associating collections.

You might notice the lack of `NSList` or `NSQueue`. Indeed, it seems that these concepts should be implemented as mere `NSArray` objects.

Unlike a `vector<T>` in C++, an `NSArray` of Objective-C really hides its internal implementation and only exposes its content through the accessors. Therefore, an `NSArray` has no obligation to organize its content as contiguous memory cells. The implementors of `NSArray` have certainly chosen a compromise, letting `NSArray` be used efficiently like an array or like a list. Since in Objective-C a container can only hold pointers to objects, the manipulation of the cells can be very efficient.

11.2 Iterators

11.2.1 Classical enumeration

The pure object-oriented approach makes the notion of iterator more flexible in Objective-C than in C++. The `NSEnumerator` is designed so that:

```
NSArray* array = [NSArray arrayWithObjects:object1, object2, object3, nil];
NSEnumerator* enumerator = [array objectEnumerator];
NSString* aString = @"foo";
id anObject = [enumerator nextObject];
while (anObject != nil)
{
    [anObject doSomethingWithString:aString];
    anObject = [enumerator nextObject];
}
```

A method of the container (`objectEnumerator`) returns an iterator, which is then able to move by itself (`nextObject`). The behaviour is closer to Java than to C++. When the iterator reaches the end of the container, `nextObject` returns `nil`.

There is a more common syntax to use iterators, that is based only on the C ability to shorten things.

```
NSArray* array = [NSArray arrayWithObjects:object1, object2, object3, nil];
NSEnumerator* enumerator = [array objectEnumerator];
NSString* aString = @"foo";
id anObject = nil;
while ((anObject = [enumerator nextObject])) {
    [anObject doSomethingWithString:aString];
}
//double parenthesis suppress a gcc warning here
```

11.2.2 Fast enumeration

Objective-C 2.0 (cf. Section 15 on page 50) has introduced a new syntax to enumerate a container, so that the `NSEnumerator` is implicit (besides, it is no more an `NSEnumerator`). The syntax is of the form:

```
NSArray* someContainer = ...;
for(id object in someContainer) //here, each object is typed "id"
{
    ...
}
```

11.3 Functors (function objects)

11.3.1 Using selectors

The power of the `selector` in Objective-C makes functors less useful. Indeed, weak typing allows the user to send a message without really taking care of the ability of the receiver to handle it. For instance, here is a code equivalent to the previous one using iterators:

```
NSArray* array = [NSArray arrayWithObjects:object1, object2, object3, nil];
NSString* aString = @"foo";
[array makeObjectsPerformSelector:@selector(doSomethingWithString:)
    withObject:aString];
```

in this case, the objects don't have to be of the same kind of class, and they don't even have to implement a `doSomethingWithString:` method (it would just raise an exception "selector not recognized") !

11.3.2 IMP caching

This won't be detailed here, but it is possible to get the address of a C function which represents a method into memory. This can be used to optimize multiple calls to the same `selector` on a bunch of objects, by doing the method lookup only once. This is called "IMP caching" since `IMP` is in Objective-C the data type of a method implementation.

11.4 Algorithms

The large set of algorithms of the STL really have no equivalent in Cocoa. Instead, you should look at the methods supplied by each container.

12 C++ specific features

So far, you’ve seen that object-oriented concepts of C++ are present in Objective-C. However, some other features are not. It does not concern object-oriented concepts, but some coding facilities.

12.1 References

References (&) do not exist in Objective-C. Memory management using reference counting and `autorelease` makes it less useful. Since the objects are always dynamically created, they are only referenced as pointers.

12.2 Inlining

Objective-C does not implement inlining. This is rather logical for the methods, because the dynamism of Objective-C cannot afford to “freeze” some code. However, inlining would be very useful for utility functions written in simple C, like `max()`, `min()`... This is a problem that Objective-C++ can address.

Nevertheless, note that the GCC compiler supplies the non-standard keyword `__inline` or `__inline__` to use inlining in C, and so in Objective-C. Moreover, GCC is also able to compile C99 code, a revision of C that supports inlining with the keyword `inline` (standard in this case). Thus, Objective-C based on C99 code benefits from inlining.

This is not inlining, but for performance, you can also consider using IMP-caching (cf. section 11.3.2 on the preceding page).

12.3 Templates

Templates are a kind of alternative to inheritance and virtual methods, designed for efficiency, but clearly far from pure object model principles. (Did you know that a subtle use of templates can give `public` access to `private` members?). Objective-C does not implement templates, and it would be very difficult because of the rules about method names overloading and selectors.

12.4 Operators overloading

Objective-C does not implement operator overloading.

12.5 Friends

There is no *friend* notion in Objective-C. Indeed, this is rather useful in C++ for the efficiency of overloaded operators, that are not present in Objective-C. The notion of packages in Java, that is somewhat close to friends, can be addressed most of the time in Objective-C with class categories (cf. Section 6.5 on page 22).

12.6 const methods

Methods cannot be declared `const` in Objective-C. Therefore, the `mutable` keyword does not exist.

12.7 List of initialization in the constructor

Lists of initialization at construction does not exist in Objective-C.

13 RTTI (Run-Time Type Information)

The C++ language might be said to be a “false” object-oriented language. Compared to Objective-C, it is very static. This is a deliberate choice in favour of best performances at run-time. The run-time information that C++ can supply, through the *typeid* library, cannot be safely used because they depend on the compiler. Asking for the class of an object is a rare request because of strong typing; but it can occur when browsing a container. The `dynamic_cast` operator and, sometimes, `typeid`, can be used, but the interaction with the user of the program is limited. How to test that an object is an instance of a class known by its name ?

Objective-C is naturally designed for such questions. The classes being objects, they inherit their behaviour.

Downcasting is detailed in section 5.4.4 on page 18 since it is a particular use of `dynamic_cast`.

13.1 `class`, `superclass`, `isMemberOfClass`, `isKindOfClass`

The ability for an object to know its own type at run-time is called *introspection*, and can be handled with several methods.

`isMemberOfClass`: is a method that answers the question: “Am I an instance of a given class (leaving aside inheritance)” ?, while `isKindOfClass`: answers “Am I an instance of a given class or one of its subclasses ?”.

Using these methods requires the false keyword: `class` (and not `@class`, which is used for forward declarations). Indeed, `class` is a method of `NSObject`, and returns a `Class` object, which is an instance of meta-class. Please note that the nil value for a class pointer is not `nil` but `Nil`.

```
BOOL test = [self isKindOfClass:[Foo class]];
if (test)
    printf("I am an instance of the Foo class\n");
```

Please also note that there is a way to get a mother-class class pointer: you can use the method `superclass`.

13.2 `conformsToProtocol`

This method has been explained in the section detailing protocols (section 6.4 on page 20). It is useful to know whether an object conforms to a protocol or not. This is not really dynamic, since the compiler only checks for explicit conformance rather than checking every methods. If an object implements all the methods of a given protocol, but does not explicitly say that it conforms to that protocol, the program is correct but `conformsToProtocol`: returns `NO`.

13.3 `respondsToSelector`, `instancesRespondToSelector`

`respondsToSelector`: is a instance method, inherited from `NSObject`. It is able to check whether an object implements a given method or not. The notion of *selector* is used (cf. section 5.3.4 on page 13). For example:

```
if ( [self respondsToSelector:@selector(work)] )
{
    printf("I am not lazy.\n");
    [self work];
}
```

To know whether a class implements a given method, without checking for inherited methods, one can use the *class* method `instancesRespondToSelector:`.


```

if ([[self class] instancesRespondToSelector:@selector(findWork)])
{
    printf("I can find a job without the help of my mother\n");
}

```

Please note that a call to `respondToSelector:` cannot determine whether a class can handle a message through *forwarding* (cf. section 5.4.3 on page 17).

13.4 Strong typing or weak typing with `id`

C++ uses strong typing: one can use an object according to its apparent type only, otherwise the program cannot be compiled. In Objective-C, the constraint is more flexible if an object **whose type is explicitly known** is the target of a message that it cannot handle *at first sight*, then the compiler outputs a warning, but the program will run. The message will be lost (raising an exception), except if *forwarding* is triggered (cf. section 5.4.3 on page 17). If it is what the developer had in mind, the warning is redundant; in this case, typing the target to `id` instead of its real type removes the warning. In fact, any object is of type `id`, and can be the target of any message.

This weak-typing property is essential when using delegation: the delegate object does not have to be known to be used. Here is an example:

```

-(void) setAssistant:(id)anObject
{
    [assistant autorelease];
    assistant = [anObject retain];
}

-(void) manageDocument:(Document*)document
{
    if ([assistant respondsToSelector:@(manageDocument:)])
        [assistant manageDocument:document];
    else
        printf("Did you fill the blue form ?\n");
}

```

Delegations are very common in Cocoa when a graphical interface is used, so controls can transmit actions from the user to worker objects.

14 Objective-C++

Objective-C++ is still being developed. It is usable, but still has some gaps. Objective-C++ allows the concurrent use of Objective-C and C++, to take the best of both.

When mixing C++ and Objective-C exceptions, or when using C++ objects as instance data of Objective-C objects, refer to the documentation in order to understand the expected behaviour. However, it is definitely possible to write programs in Objective-C++. The implementation files have the extension `.mm`.

Since MacOS X 10.4, a C++ object can be used as instance data of an Objective-C class, and benefit from automatic construction/destruction ¹.

¹<http://developer.apple.com/releasenotes/Cocoa/RN-ObjectiveC/index.html>

15 Objective-C 2.0

The revision 2.0 of Objective-C has introduced some new features. The current highlights include:

- the garbage collector (cf. section 8.6 on page 41);
- fast enumerations (cf. section 11.2.2 on page 46);
- the `@optional` keyword for protocols;
- *properties*.

The concept of *properties* for instance data, makes it reachable with a structure-like syntax, using the dot (“.”), without explicit use of getters and setters. This is not only about syntax. Please read the Objective-C 2.0 documentation to learn more.

Conclusion

This document is a quick look at many aspects of Objective-C, compared to C++ notions. It is useful as a quick reference for advanced developers willing to learn the language. I hope that the goal has been reached, but the reader is still welcome to provide any feedback to improve it again.

References

- [1] Apple Computer, Inc., Developer documentation. The Objective-C Programming Language for MacOS X version 10.3, February 2004. <http://developer.apple.com/documentation/Cocoa/Conceptual/ObjectiveC/ObjC.pdf>.
- [2] Boost. <http://www.boost.org/>.
- [3] GNUstep. <http://www.gnustep.org/>.
- [4] Aaron Hillegass. *Cocoa Programming for MacOS X, 2nd edition*. Addison-Wesley, 2004.
- [5] SGI. Standard template library programmer's guide. <http://www.sgi.com/tech/stl>.
- [6] Will Shipley. `self = [supid init];` <http://wilshipley.com/blog/2005/07/self-stupid-init.html>.

Document revisions

version 1.11

- fixed typos.

version 1.10

- english adaptation by Aaron VEGH;
- the section about cloning has been rewritten;
- added a section about IMP caching;
- precisions on `initialize`;
- precisions on Objective-C++;
- fixed error about `superclass`;
- precisions about “static” instance data;
- precisions on the labels of method parameters;
- fixed typos about virtual constructors;
- fixed mistakes about `self` and `super`;
- precisions about `in`, `out`, `byref...`
- precisions about Objective-C 2.0;
- precisions about static strings;
- fixed minor typos.

version 1.9

- minor changes.

version 1.8

- first english version.

Index

- .h, **8**
- .m, **8**
- .mm, **8**, **49**
- @class, **10**
- @protocol, **10**
- #import, **8**
- #include, **8**
- %, **44**
- Objective-C++, **8**, **49**
- Objective-C++ 2.0, **50**

- accessor
 - mutating, **37**
 - reading, **39**
- algorithms, **46**
- alloc, **24**, **33**
- arguments
 - anonymous, **16**
 - default value, **16**
 - variable number, **16**
- attributes, **9**
 - static, **11**
- autorelease, **26**, **33**, **35**
 - abusive use, **35**
 - pool, **35**

- BOOL (type), **7**
- bycopy, **22**
- byref, **22**

- catÃ©gorie de classe, **22**
- catch, **42**
- class, **37**, **48**
- class categories, **19**, **23**
- class category, **11**, **21**
- class data, **11**
- classes, **9**
 - NS classes, **7**
 - root class, **6**, **9**
- classical enumeration, **45**
- Cocoa, **6**, **7**, **45**
- comments, **7**
- const
 - method, **17**
 - methods, **47**
- constructor, **24**
 - class constructor, **28**
 - convenience constructor, **36**
 - default, **26**
 - list of initialization, **28**, **47**
 - virtual, **28**, **36**
- container, **45**
- copy, **29**, **33**
 - mutable, immutable, **31**
 - operator, **29**
 - copyWithZone, **29**

- delegation, **17**
- delete, **33**
- destructors, **28**
- document
 - revisions, **52**
- downcasting, **18**
- dynamic_cast, **18**, **48**

- encodage, **44**
- enumeration
 - classical, **45**
 - fast, **46**
- exceptions, **42**
 - catch, **42**
 - finally, **42**
 - throw, **42**
 - try, **42**

- fast enumeration, **46**
- fichiers
 - .mm, **49**
 - implementation, **8**
- files
 - .h, **8**
 - .m, **8**
 - .mm, **8**
 - header, **8**
 - inclusion, **8**
- finally, **42**
- forward declaration, **10**
- forwarding, **17**
- friend, **17**, **47**
- function-object, **46**
- functions, **8**
- functor, **46**

- garbage collector, **41**
- getter, **39**

- history
 - Objective-C, **6**
 - document, **52**

- id, **7**, **9**, **49**
- IMP caching, **46**
- in, **22**
- inclusion of files, **8**
- inheritance
 - multiple, **19**, **20**
 - public, protected, private, **19**
 - simple, **19**

- virtual, 19
- inheritance , **19**
- init, **24**
- initialize, 28
- initializer, **24**
 - designated, 26
- inline, **47**
- inout, **22**
- instance data, **9**
- introspection, **48**
- isKindOfClass, **48**
- isMemberOfClass, **48**
- iterators, **45**
 - classical enumeration, 45
 - fast enumeration, 46
- keywords of Objective-C, **6**
- list of initialization, **28**
- memory, **33**
 - alloc, 33
 - autorelease, 33
 - copy, 33
 - custom zones, 29
 - delete, 33
 - mutable, non-mutable copy, 31
 - mutableCopy, 33
 - new, 33
 - reference counting, 33
 - release, 33
 - retain, 33
- method
 - class method, 17
 - const, 17
 - static, 17
- methods, 8, **9**
 - de classe, 11
 - pure virtual, 17, 19, 20
 - virtuals, 19
- mutable, 17
 - keyword, **47**
- mutableCopy, **31**, 33
- mutableCopyWithZone, **31**
- mutator, **37**
- new, **33**
- Nil, 7, **9**, 48
- nil, 7, **9**, 17
- NSList, **45**
- NSQueue, **45**
- object
 - function, 46
 - mutable, immutable, 31
- oneway, **22**
- out, **22**
- overloading
 - functions, 13
 - methods, 13
 - operators, 47
- parameters
 - anonymous, 16
 - default value, 16
 - variable number, 16
- pointer to member function, **14**
- private, **11**
 - inheritance, 19
- properties, 50
- protected, **11**
 - inheritance, 19
- protocol, **20**
 - conformsToProtocol, 48
 - formal, 20
 - informal, 21
 - qualifiers
 - in, out, inout, bycopy, byref, oneway, 22
- protocole, 23
- prototype, **11**, 13
 - modifiers, 17
- public, **11**
 - inheritance, 19
- qualifiers
 - in, out, inout, bycopy, byref, oneway, 22
- ramasse-miettes, 41
- references, **47**
- release, **33**
- respondsToSelector, **48**
- retain, **33**
- retain cycle, 40
- revisions
 - document, **52**
- root class, 6
- RTTI, **48**
- sÃ©lecteur
 - respondsToSelector, 48
- SEL (type), **7**
- selector, **14**
 - pointer to member function, 14
 - type SEL, 7
- self, **12**
- setter, **37**
- static, 11, **17**
- STL, **45**
 - algorithms, 46
 - containers, 45
 - functors, 46
 - iterators, 45
- strings, **44**
- super, **12**

- superclass, **48**
- templates, **47**
- this, **12**
- throw, 17, **42**
- try, **42**
- type
 - BOOL, 7
 - id, 7, **9**
 - SEL, 7
- virtual, 17, **19**
 - inheritance, 19
 - constructor, 36
 - methods, 19
 - pure virtual methods, 17, 19, 20